

8 ПРАКТИЧЕСКАЯ РАБОТА (РАЗРАБОТКА ФУНКЦИЙ, ХРАНИМЫХ ПРОЦЕДУР И ТРИГГЕРОВ)

8.1 Цель работы

Целью практической работы является формирование навыков разработки серверной логики системы управления данными средствами реляционной СУБД, включая создание пользовательских функций, хранимых процедур и триггеров для реализации вычислений, проверок, бизнес-операций, автоматического изменения данных и журналирования событий.

8.2 Задачи работы

В процессе выполнения практической работы необходимо:

- проанализировать требования проектируемой системы и ранее разработанные SQL-запросы;
- определить бизнес-операции, целесообразные для реализации на стороне СУБД;
- определить задачи, для решения которых могут применяться пользовательские функции;
- определить операции, для которых целесообразно использование хранимых процедур;
- определить события базы данных, требующие автоматической обработки средствами триггеров;

- разработать не менее двух серверных объектов из числа функций и хранимых процедур;
- разработать не менее двух содержательных триггеров;
- обеспечить передачу параметров в функции и процедуры;
- реализовать обработку корректных и ошибочных сценариев;
- проверить влияние серверной логики на состояние базы данных;
- исключить дублирование одной и той же бизнес-логики в нескольких независимых местах;
- оформить разработанные объекты в виде воспроизводимых SQL-скриптов;
- подготовить серверную логику к последующему использованию в пользовательском интерфейсе.

8.3 Теоретические сведения

Современные реляционные СУБД позволяют размещать часть программной логики непосредственно в базе данных.

К основным средствам реализации серверной логики относятся:

- хранимые процедуры;
- пользовательские функции;
- триггеры.

Использование серверной логики позволяет централизовать операции над данными и обеспечить выполнение определённых правил независимо от того, какое прикладное программное обеспечение обращается к базе данных.

При этом перенос логики на уровень СУБД должен быть обоснован.

Не каждая операция системы должна реализовываться посредством процедуры, функции или триггера.

В рамках проектирования необходимо определить, какой уровень системы является наиболее подходящим для реализации конкретного правила:

- декларативные ограничения базы данных;
- функция;
- процедура;
- триггер;
- программный код приложения.

8.3.1 Хранимые процедуры

Хранимая процедура представляет собой блок программного кода, сохраняемый непосредственно в базе данных и выполняемый средствами СУБД.

Хранимые процедуры могут использоваться повторно и вызываться:

- из прикладной программы;
- из других серверных объектов;
- непосредственно средствами СУБД.

Процедура может реализовывать последовательность взаимосвязанных операций над данными.

К типовым задачам хранимых процедур относятся:

- изменение состояния объекта;
- регистрация сложной операции;
- выполнение нескольких связанных изменений;
- формирование набора данных;

- выполнение бизнес-операции;
- централизованное выполнение повторяющейся логики.

Например, вместо выполнения приложением нескольких независимых SQL-команд процедура может реализовывать единую операцию:

зарегистрировать заказ, создать позиции заказа и сформировать первоначальное состояние заказа.

Такой подход позволяет сосредоточить связанную бизнес-логику в единой точке.

8.3.2 Параметры хранимых процедур

Хранимые процедуры могут принимать входные параметры.

Параметры позволяют использовать одну процедуру для различных значений.

Например:

- идентификатор пользователя;
- идентификатор заказа;
- новое состояние;
- дата;
- сумма;
- категория.

В зависимости от возможностей конкретной СУБД могут также использоваться выходные параметры.

Синтаксис создания и вызова хранимых процедур различается в разных СУБД.

Поэтому при реализации необходимо использовать синтаксис, соответствующий выбранной в ПР №6 системе управления базами данных.

8.3.3 Назначение хранимых процедур в проекте

Процедура должна представлять самостоятельную операцию предметной области.

Примерами содержательных процедур могут являться:

- назначение исполнителя на заявку;
- оформление возврата;
- проведение платежа;
- закрытие заказа;
- перевод объекта в новое состояние;
- регистрация выдачи оборудования;
- формирование операции списания;
- отмена бронирования;
- создание комплексной записи, затрагивающей несколько таблиц.

Не рекомендуется создавать процедуры вида:

AddUser, содержащую единственную команду `INSERT INTO users ...`

если такая процедура не реализует дополнительную бизнес-логику и практически полностью дублирует обычную SQL-команду.

Простая операция может быть реализована процедурой, если это необходимо архитектуре проекта, однако в рамках минимальных требований настоящей работы она не рассматривается как достаточно содержательная без дополнительного обоснования.

8.3.4 Пользовательские функции

Пользовательская функция представляет собой программный объект базы данных, принимающий входные параметры и возвращающий результат.

Функции могут использоваться для:

- выполнения вычислений;
- преобразования данных;
- получения производного значения;
- формирования результирующего набора данных;
- повторного использования логики в SQL-выражениях.

Функция должна иметь определённый возвращаемый результат.

В зависимости от СУБД могут поддерживаться:

- скалярные функции;
- табличные функции;
- иные разновидности пользовательских функций.

8.3.5 Скалярные функции

Скалярная функция возвращает одно значение.

Например, функция может определять:

- итоговую сумму заказа;
- размер скидки;
- количество просроченных объектов;
- текущее значение вычисляемого показателя;
- продолжительность выполнения операции;

- категорию объекта на основании набора признаков.

Пример логической задачи:

Рассчитать итоговую сумму заказа по его идентификатору с учётом всех позиций.

В этом случае вызывающая сторона получает одно результирующее значение.

8.3.6 Табличные функции

Табличная функция возвращает набор строк.

Она может применяться, например, для получения:

- списка активных заявок определённого пользователя;
- списка сотрудников выбранного подразделения;
- набора просроченных заказов;
- истории операций по объекту;
- результатов по заданному диапазону значений.

Использование табличной функции должно быть оправдано повторным использованием соответствующей выборки или необходимостью логически выделить её в отдельный объект.

8.3.7 Отличие функции от процедуры

Функция и процедура являются серверными программными объектами, однако решают разные задачи.

В общем случае функция ориентирована на получение определённого результата и может использоваться как часть вычислений или запросов.

Хранимая процедура ориентирована на выполнение законченной последовательности действий и может изменять состояние базы данных.

Конкретные возможности функций и процедур зависят от выбранной СУБД.

Поэтому при проектировании следует учитывать ограничения конкретной реализации.

Не рекомендуется выбирать тип объекта только на основании удобства синтаксиса.

Необходимо исходить из назначения выполняемой операции.

8.3.8 Триггеры

Триггер представляет собой специальный программный объект, автоматически выполняемый при наступлении определённого события в базе данных.

В отличие от процедуры, триггер не вызывается пользователем или приложением непосредственно.

Его выполнение инициируется событием.

Наиболее распространёнными событиями являются:

- INSERT;
- UPDATE;
- DELETE.

Триггеры могут использоваться для реализации дополнительной логики, связанной с изменением данных.

Например:

- автоматическое журналирование изменений;
- контроль сложного бизнес-правила;
- обновление связанной информации;
- автоматическое изменение состояния;
- предотвращение недопустимой операции;
- фиксация истории изменения объекта.

В зависимости от СУБД и характера события могут использоваться различные моменты выполнения триггера.

BEFORE-триггер выполняется до основной операции.

Он может использоваться, например, для:

- проверки данных;
- подготовки или преобразования значений;
- предотвращения недопустимой операции.

AFTER-триггер выполняется после завершения основной операции.

Он может использоваться, например, для:

- журналирования;
- регистрации факта изменения;
- выполнения связанного действия;
- обновления зависимой информации.

Выбор момента выполнения триггера должен соответствовать смыслу реализуемого правила.

Триггер должен быть связан с конкретным событием или набором событий.

Типичными являются:

INSERT Триггер выполняется при создании новой записи.

Возможные задачи:

- создание записи истории;

- первоначальная установка связанных данных;
- автоматическое журналирование создания объекта.

UPDATE Триггер выполняется при изменении существующих данных.

Возможные задачи:

- фиксация старого и нового состояния;
- проверка допустимости перехода;
- изменение связанных данных;
- регистрация времени изменения.

DELETE Триггер выполняется при удалении данных.

Возможные задачи:

- журналирование удаления;
- предотвращение удаления значимого объекта;
- сохранение необходимой истории;
- выполнение сопутствующих действий.

8.3.9 Требования к применению триггеров

Триггер должен использоваться только в тех случаях, когда автоматическая реакция базы данных на событие действительно является необходимой.

Не рекомендуется использовать триггер для логики, которую проще и прозрачнее реализовать:

- ограничением CHECK;
- внешним ключом;
- NOT NULL;
- UNIQUE;

- обычной процедурой;
- явной командой приложения.

Например, правило:

количество товара должно быть больше нуля

целесообразно реализовать посредством CHECK, а не триггером.

Правило:

при изменении состояния заявки необходимо автоматически сохранить предыдущий статус, новый статус, время изменения и пользователя в журнале истории

является более естественным примером применения триггера.

Особенностью триггеров является неявный характер их вызова.

Пользователь выполняет, например:

UPDATE ...

однако в результате могут выполняться дополнительные действия.

Это повышает возможности базы данных, но одновременно усложняет анализ поведения системы.

Поэтому каждый триггер должен:

- иметь однозначное назначение;
- быть документирован;
- выполнять ограниченный объём логики;
- не дублировать другой триггер;
- не создавать неочевидных циклических зависимостей.

При разработке необходимо учитывать возможность последовательного срабатывания нескольких триггеров.

8.3.10 Выбор средства реализации бизнес-правила

Перед созданием серверного объекта необходимо определить наиболее подходящий механизм.

Рекомендуется использовать следующую общую логику.

Характер задачи	Предпочтительное средство
Обязательность значения	NOT NULL
Уникальность	UNIQUE
Простая проверка диапазона	CHECK
Ссылочная целостность	FOREIGN KEY
Вычисление повторно используемого значения	Функция
Получение повторно используемого набора данных	Функция
Сложная явно вызываемая бизнес-операция	Процедура
Несколько связанных изменений	Процедура
Автоматическая реакция на изменение данных	Триггер
Журналирование изменения	Триггер

Данная таблица представляет общую рекомендацию.

Конкретное решение определяется возможностями выбранной СУБД и архитектурой проекта.

8.3.11 Транзакционная целостность серверных операций

Если одна бизнес-операция включает несколько связанных изменений, необходимо учитывать возможность частичного выполнения.

Например, операция оформления заказа может включать:

- создание заказа;
- создание нескольких позиций;
- изменение доступного количества товара;
- регистрацию операции.

Если часть действий выполнена, а последующее действие завершилось ошибкой, база данных может перейти в некорректное состояние.

Поэтому связанные изменения должны рассматриваться как единая логическая операция.

Возможности управления транзакциями зависят от конкретной СУБД и используемого типа программного объекта.

В рамках настоящей практической работы обучающийся должен уметь объяснить, каким образом обеспечивается согласованность данных при ошибочном выполнении серверной операции.

8.3.12 Обработка ошибочных сценариев

Для каждого разработанного серверного объекта необходимо определить не только корректный, но и ошибочный сценарий.

Например:

Корректный сценарий:

заявка находится в состоянии «Новая», назначается существующий исполнитель, имеющий соответствующие полномочия.

Ошибочный сценарий:

выполняется попытка назначить исполнителя закрытой заявке.

Серверная логика должна:

- предотвратить нарушение бизнес-правила;
- сохранить базу данных в согласованном состоянии;
- по возможности сформировать понятную диагностическую информацию.

Не допускается считать объект проверенным только потому, что он один раз успешно выполнен на корректных данных.

8.4 Задание

На основании требований, физической структуры базы данных и SQL-запросов необходимо определить и реализовать серверную бизнес-логику проектируемой системы.

Работа должна включать:

- пользовательские функции и/или хранимые процедуры;
- триггеры;
- сценарии проверки разработанных объектов.

8.4.1 Анализ бизнес-правил

Перед непосредственной разработкой необходимо сформировать перечень бизнес-правил и операций, которые потенциально могут быть реализованы на стороне СУБД.

Рекомендуется использовать таблицу:

Правило / операция	Источник требования	Выбранный механизм	Обоснование
Расчёт итоговой стоимости заказа	FR-...	Функция	Повторно используемое вычисление
Назначение исполнителя	FR-...	Процедура	Связанная бизнес-операция
Сохранение истории изменения состояния	BR-...	Триггер	Автоматическая реакция на UPDATE

Необходимо показать, что выбор конкретного типа серверного объекта является результатом анализа.

8.4.2 Разработка функций и процедур

Необходимо реализовать не менее 2 содержательных серверных объектов из числа пользовательских функций и хранимых процедур.

При наличии технической возможности выбранной СУБД рекомендуется, чтобы набор включал:

- не менее 1 пользовательской функции;
- не менее 1 хранимой процедуры.

Если архитектура проекта или особенности СУБД обосновывают иной состав, он должен быть согласован с преподавателем.

Объекты должны решать различные задачи.

Не допускается представлять два практически одинаковых объекта, отличающихся только названием таблицы или одним параметром.

Как минимум один из разработанных объектов должен обеспечивать содержательную работу с данными.

Для функций это может быть получение или вычисление данных.

Для процедур — изменение данных либо выполнение законченной бизнес-операции.

8.4.3 Разработка триггеров

Необходимо реализовать не менее 2 содержательных триггеров.

Триггеры должны решать различные задачи либо реагировать на различные бизнес-события.

Рекомендуемые направления:

- журналирование;

- контроль допустимости переходов состояния;
- фиксация истории;
- автоматическое формирование связанных данных;
- поддержание дополнительной согласованности данных.

Не допускается учитывать в минимальном количестве триггеры, которые не выполняют фактически значимой логики.

Например:

AFTER INSERT-триггер, который записывает в другую таблицу только текст «добавлена запись»

без связи с задачами проектируемой системы может быть признан формальным.

8.4.4 Требование различия серверных объектов

Разрабатываемые объекты должны демонстрировать различные механизмы работы СУБД.

Не рекомендуется реализовывать:

- две функции, выполняющие одинаковую выборку;
- две процедуры с одинаковой структурой над разными таблицами;
- два триггера, записывающие один и тот же тип сообщения в журнал.

Минимальный набор должен показывать, что обучающийся понимает различие между:

- явным вызовом;
- вычислением;
- бизнес-операцией;
- автоматической реакцией на событие.

8.4.5 Параметры

Функции и процедуры должны использовать параметры там, где это соответствует назначению объекта.

Не допускается жёстко задавать внутри объекта идентификатор конкретной тестовой записи только для демонстрации работы.

Например, вместо:

обработать заказ № 5

объект должен принимать идентификатор заказа параметром.

Это необходимо для последующего использования серверной логики пользовательским интерфейсом.

8.4.6 Проверка функции

Для каждой пользовательской функции необходимо:

- определить входные параметры;
- определить возвращаемый результат;
- выполнить функцию на корректных данных;
- проверить правильность результата;
- проверить граничные или особые случаи;
- при необходимости проверить поведение при отсутствии данных.

8.4.7 Проверка процедуры

Для каждой хранимой процедуры необходимо:

- определить входные параметры;
- определить ожидаемое изменение базы данных;
- зафиксировать исходное состояние;
- выполнить процедуру;
- проверить полученный результат;
- проверить все изменённые таблицы;
- выполнить ошибочный сценарий;
- убедиться в отсутствии частично выполненной некорректной операции.

8.4.8 Проверка триггера

Для каждого триггера необходимо:

- определить событие срабатывания;
- определить момент выполнения;
- определить ожидаемое автоматическое действие;
- зафиксировать состояние базы до операции;
- выполнить исходную операцию;
- проверить факт срабатывания;
- проверить результирующее состояние базы;
- выполнить альтернативный или ошибочный сценарий.

Если триггер предназначен для предотвращения операции, необходимо показать, что:

- недопустимая операция действительно блокируется;
- данные остаются согласованными.

8.4.9 SQL-скрипты серверной логики

Разработанные объекты должны быть представлены в воспроизводимых SQL-скриптах.

Рекомендуемая структура:

/sql
01_create_tables.sql
02_insert_data.sql
03_queries.sql
04_functions.sql
05_triggers.sql

Если выбранная СУБД требует отдельного файла процедур, допускается структура:

/sql
04_functions.sql
05_procedures.sql
06_triggers.sql

Нумерация последующих файлов в этом случае должна быть скорректирована последовательно.

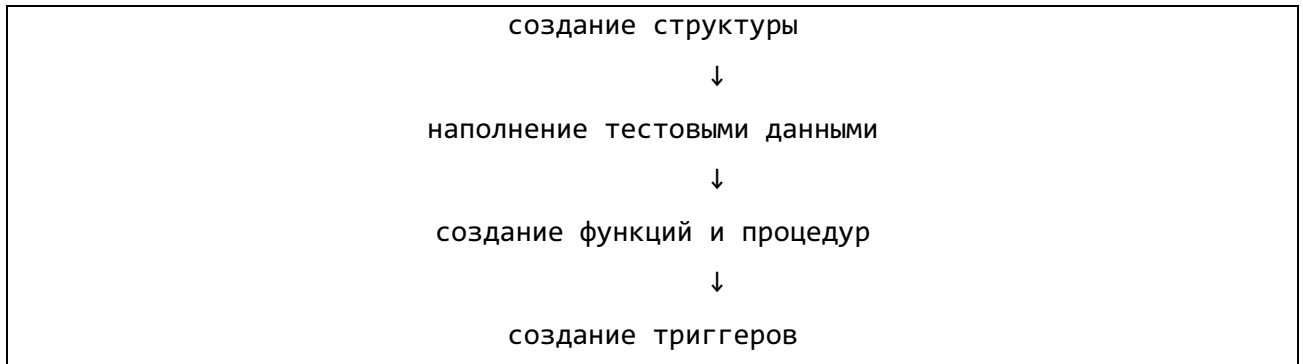
Каждый объект должен сопровождаться комментарием, содержащим:

- назначение;
- связанное требование или бизнес-правило;
- краткое описание результата.

8.4.10 Требования к воспроизводимости

Серверная логика должна создаваться посредством SQL-скриптов.

После развёртывания новой базы должна существовать возможность выполнить последовательно:



и получить базу данных, содержащую все разработанные серверные объекты.

Создание функции, процедуры или триггера исключительно вручную через графический интерфейс СУБД без соответствующего SQL-скрипта не считается завершённой реализацией.

8.5 Минимальные требования к результату

Для зачёта практической работы необходимо обеспечить:

- не менее 2 содержательных объектов из числа функций и хранимых процедур;
- не менее 2 содержательных триггеров;
- использование входных параметров в функциях и/или процедурах там, где это необходимо;
- решение различающихся прикладных задач;
- связь каждого объекта с требованиями или бизнес-правилами системы;

- наличие как минимум одного объекта, выполняющего содержательную операцию над данными;
- наличие как минимум одного автоматического сценария обработки изменения данных;
- проверку корректных сценариев;
- проверку ошибочных или граничных сценариев;
- воспроизводимые SQL-скрипты создания всех объектов.

Соблюдение минимального количества объектов не является достаточным условием зачёта при отсутствии их содержательного назначения.

8.6 Проверка согласованности с предыдущими практическими работами

Перед завершением работы необходимо проверить:

- функции и процедуры используют существующую структуру БД;
- бизнес-правила соответствуют требованиям ПР №1–2;
- операции не противоречат моделям бизнес-процессов;
- используемые объекты данных присутствуют в логической и физической моделях;
- серверная логика не дублирует существующие ограничения без необходимости;
- ранее разработанные SQL-запросы остаются корректными;
- изменения структуры отражены в 01_create_tables.sql;
- новые необходимые тестовые данные отражены в 02_insert_data.sql.

Если для реализации серверной логики требуется изменение физической структуры, необходимо актуализировать ПР №5–7.

8.7 Порядок выполнения работы

- Проанализировать замечания по ПР №7.
- Составить перечень бизнес-правил и операций.
- Определить правила, уже реализованные декларативными ограничениями.
- Выделить задачи для пользовательских функций.
- Выделить задачи для хранимых процедур.
- Выделить события для триггеров.
- Обосновать выбор механизма для каждого объекта.
- Разработать первую функцию или процедуру.
- Проверить корректный сценарий.
- Проверить граничный или ошибочный сценарий.
- Разработать второй серверный объект.
- Выполнить его проверку.
- Разработать первый триггер.
- Проверить событие срабатывания.
- Разработать второй триггер.
- Проверить его работу.
- Проверить совместное влияние серверных объектов.
- Исключить дублирование и конфликтующую логику.
- Оформить SQL-скрипты.

- Проверить развёртывание объектов в новой базе.
- Подготовить демонстрационные запросы.
- Подготовить отчёт.
- Подготовить результаты к защите.

8.8 Содержание отчёта

Отчёт по практической работе должен содержать:

- титульный лист;
- номер и название практической работы;
- цель работы;
- перечень выбранных бизнес-правил и операций;
- обоснование распределения логики между функциями, процедурами, триггерами и ограничениями;
- описание каждой разработанной функции;
- описание каждой разработанной процедуры;
- описание каждого разработанного триггера;
- входные параметры серверных объектов;
- события срабатывания триггеров;
- основные фрагменты SQL-кода;
- примеры вызова функций и процедур;
- примеры операций, вызывающих триггеры;
- результаты корректных сценариев;
- результаты ошибочных или граничных сценариев;
- описание изменений состояния базы данных;

- сведения об актуализации предыдущих материалов;
- вывод по результатам работы.

Для каждого серверного объекта рекомендуется использовать таблицу:

Параметр	Описание
Наименование	Функция / процедура / триггер
Тип объекта	
Назначение	
Связанное требование	
Входные параметры	
Возвращаемый результат / изменение	
Событие запуска	
Ошибочный сценарий	

8.9 Результаты, предоставляемые преподавателю

Для защиты практической работы обучающийся предоставляет:

- отчёт;
- актуальный 01_create_tables.sql;
- актуальный 02_insert_data.sql;
- SQL-скрипты функций;
- SQL-скрипты процедур, если выделены отдельно;
- SQL-скрипты триггеров;
- развернутую тестовую базу либо возможность её воспроизвести.

При защите обучающийся должен быть готов:

- объяснить назначение каждого серверного объекта;
- указать требование или бизнес-правило, которое он реализует;
- объяснить выбор между функцией, процедурой и триггером;
- показать входные параметры;
- вызвать функцию или процедуру с различными параметрами;

- самостоятельно сформировать операцию, вызывающую триггер;
- показать состояние данных до и после выполнения;
- продемонстрировать ошибочный сценарий;
- объяснить поведение базы данных при возникновении ошибки.

8.10 Использование результатов в курсовой работе

Разработанная серверная логика является частью программно-информационной реализации курсовой работы.

Минимальные требования настоящей практической работы:

- 2 функции/процедуры;
- 2 триггера

определяют только объём, достаточный для зачёта ПР №8.

В курсовой работе количество серверных объектов должно определяться реальной логикой проектируемой системы.

Если система содержит десять бизнес-операций, для которых обоснованно требуется серверная реализация, наличие только минимальных четырёх объектов не означает завершённость соответствующей части проекта.

В итоговой системе серверная логика должна:

- соответствовать требованиям;
- обеспечивать бизнес-правила;
- не дублировать декларативные ограничения без необходимости;
- использоваться прикладным программным обеспечением;
- быть проверяемой;
- быть документированной.

Особое значение имеет связь серверной логики с GUI.

Если функция или процедура реализует пользовательскую операцию, в курсовой работе соответствующая операция должна вызываться через программный интерфейс системы, если это предусмотрено её назначением.

Триггеры должны проявляться при выполнении соответствующих пользовательских операций.

Например, если триггер сохраняет историю изменения статуса заявки, изменение статуса через GUI должно приводить к появлению соответствующей записи истории.

Следовательно, создавать серверные объекты, которые существуют только для демонстрации в SQL-консоли и никак не используются итоговой системой, для курсовой работы недостаточно.

8.11 Последствия выполнения работы только на минимальном уровне

Если в рамках настоящей практической работы обучающийся реализует простейшие функции и триггеры исключительно ради выполнения количественного минимума, при подготовке курсовой работы может потребоваться существенная переработка серверной части.

При добавлении новой бизнес-логики позднее могут потребоваться изменения:

- SQL-скриптов;
- тестовых данных;
- модели ролей;
- GUI;

- тестовых сценариев;
- технического задания;
- пояснительной записки.

Поэтому рекомендуется уже в ПР №8 выбирать серверные операции, которые будут реально использоваться в итоговой версии проекта.

8.12 Подготовка к ПР №9

После завершения серверной логики необходимо определить, какие категории пользователей имеют право:

- читать различные данные;
- изменять данные;
- выполнять процедуры;
- использовать функции;
- инициировать операции, приводящие к срабатыванию триггеров.

Эти сведения будут использованы в ПР №9 при разработке пользователей, ролей и модели разграничения доступа.

Следовательно, к моменту завершения ПР №8 перечень основных серверных операций должен быть стабилизирован.

8.13 Вспомогательные материалы

Для выбора типа объекта рекомендуется последовательно ответить на вопросы.

Вопрос 1. Можно ли реализовать правило декларативным ограничением?

Если да, предпочтительно использовать:

- NOT NULL;
- UNIQUE;
- CHECK;
- FOREIGN KEY.

Вопрос 2. Требуется получить вычисляемый результат?

Рассмотреть пользовательскую функцию.

Вопрос 3. Требуется явно выполнить законченную бизнес-операцию?

Рассмотреть хранимую процедуру.

Вопрос 4. Операция должна выполняться автоматически при изменении данных независимо от вызывающего приложения?

Рассмотреть триггер.

При проектировании триггера необходимо дополнительно определить:

- какое событие его вызывает;
- требуется ли выполнение до или после события;
- какие данные он использует;
- какие данные изменяет;
- что произойдёт при ошибке;
- может ли его выполнение вызвать другой триггер.

8.14 Контрольные вопросы

- Что представляет собой хранимая процедура?
- Какие задачи целесообразно реализовывать посредством хранимых процедур?
- Что представляет собой пользовательская функция?
- В чём заключается различие между скалярной и табличной функцией?
- В чём состоит принципиальное различие между функцией и процедурой?
- Что представляет собой триггер?
- Чем вызов триггера отличается от вызова процедуры?
- Какие события могут инициировать выполнение триггера?
- В чём заключается различие между BEFORE- и AFTER-триггером?
- В каких случаях использование триггера является обоснованным?
- Почему простые ограничения данных предпочтительно реализовывать посредством CHECK, UNIQUE, NOT NULL и FOREIGN KEY?
- Какие проблемы может создавать чрезмерное использование триггеров?
- Для чего функции и процедуры используют параметры?
- Почему серверный объект не должен быть привязан к идентификатору одной тестовой записи?
- Каким образом должна проверяться процедура, изменяющая несколько таблиц?
- Почему необходимо проверять ошибочные сценарии?
- Каким образом серверная логика будет использоваться в GUI?

- Как связаны серверные объекты и последующее разграничение доступа?
- Почему SQL-скрипты серверной логики должны быть воспроизводимыми?
- По каким признакам можно определить, что серверный объект создан формально и не имеет прикладного значения?

8.15 Критерии приёмки работы

Практическая работа может быть зачтена при одновременном выполнении следующих условий:

- проведён анализ бизнес-правил;
- обоснован выбор механизмов реализации;
- разработано не менее 2 содержательных объектов из числа функций и процедур;
- разработано не менее 2 содержательных триггеров;
- серверные объекты решают различающиеся задачи;
- используются параметры там, где они необходимы;
- присутствует содержательная работа с данными;
- триггеры действительно выполняются автоматически;
- серверная логика связана с требованиями системы;
- отсутствует необоснованное дублирование декларативных ограничений;
- проведены проверки корректных сценариев;
- проведены проверки ошибочных или граничных сценариев;

- состояние базы после выполнения операций остаётся согласованным;
- все серверные объекты воспроизводятся из SQL-скриптов;
- материалы ПР №5–7 актуализированы при необходимости;
- обучающийся способен объяснить принцип работы каждого объекта;
- обучающийся способен продемонстрировать вызов функций и процедур;
- обучающийся способен самостоятельно вызвать срабатывание триггера;
- отсутствуют объекты, созданные исключительно для формального выполнения количественных требований.

8.16 Срок выполнения и сдачи

Срок выполнения и защиты практической работы определяется календарным графиком практических занятий.

Практическая работа №8 должна быть завершена до окончательного формирования модели пользователей, ролей и прав доступа.

К моменту перехода к ПР №9 должны быть определены основные таблицы, SQL-операции, функции, процедуры и триггеры, поскольку модель разграничения доступа должна учитывать не только доступ к данным, но и возможность выполнения разработанной серверной логики.